Assignment: Earthquake Data Quality Pipeline

Objective

Build a real-world data cleaning and preprocessing pipeline using live earthquake event data from the USGS Earthquake API.

You will:

- fetch live API data
- flatten nested JSON
- clean and standardize the dataset
- handle missing values and duplicates
- create derived analytical features
- generate a final analytics-ready dataset
- create a data quality report

Data Source

Use the USGS Earthquake API:

```
https://earthquake.usgs.gov/fdsnws/event/1/query?
format=geojson&starttime=YYYY-MM-DD&endtime=YYYY-MM-
DD&minmagnitude=2.5&orderby=time
```

You may choose an appropriate date range.

Required Columns

From the API response, create a DataFrame with the following columns:

```
[
    "event_id",
    "magnitude",
    "place",
    "event_time",
    "updated_time",
    "tsunami",
    "significance",
    "alert",
    "status",
    "event_type",
```

```
    "longitude",
    "latitude",
    "depth_km"
]
```

Coordinates must come from:

```
geometry["coordinates"]
```

where:

```
longitude = coordinates[0]
latitude = coordinates[1]
depth_km = coordinates[2]
```

Data Cleaning Rules

Missing Values

Apply the following rules:

Column	Action
event_id	Drop row if missing
magnitude	Drop row if missing
place	Fill with "Unknown"
alert	Fill with "none"
latitude	Drop row if missing
longitude	Drop row if missing
depth_km	Drop row if missing

Duplicate Handling

Remove duplicate records using:

```
subset=["event_id"]
```

Data Type Conversion

Convert columns to the correct data types:

Column	Type
event_id	string
magnitude	float
place	string
event_time	datetime
updated_time	datetime
tsunami	integer
significance	integer
alert	string
status	string
event_type	string
longitude	float
latitude	float
depth_km	float

Use:

```
pd.to_datetime(..., errors="coerce")
```

for datetime conversion.

Text Cleaning

Clean the following columns:

```
place
alert
status
event_type
```

Use:

```
.str.strip()
.str.lower()
```

Also create:

```
place_clean
```

using:

```
.str.title()
```

Date Features

From `event_time`, create:

```
event_year
event_month
event_day
event_hour
```

Feature Engineering

Risk Level

Create a column:

```
risk_level
```

Rules:

High Risk

- magnitude >= 6.0
- OR tsunami == 1
- OR alert is yellow/orange/red

Medium Risk

- magnitude >= 4.5 and magnitude < 6.0
- OR significance >= 600

Low Risk

- all remaining records

Allowed values:

```
["low", "medium", "high"]
```

Depth Category

Create:

```
depth_category
```

Rules:

Condition	Category
depth_km < 70	Shallow
70 <= depth_km <= 300	Intermediate
depth_km > 300	Deep

Region Extraction

Extract a region from the `place` column.

Example:

```
"10 km S of Volcano, Hawaii" -> "Hawaii"
```

If no comma exists, use:

```
"Unknown"
```

Output Files

Save the final cleaned dataset as:

```
earthquakes_clean.csv
earthquakes_clean.parquet
```

Also save the raw API response as:

```
raw_earthquakes.json
```

Data Quality Report

Create a dictionary:

```
{
    "data_fetched_at_utc": "...",
    "raw_record_count": ...,
    "clean_record_count": ...,
    "duplicates_removed": ...,
    "missing_values_before": {...},
    "missing_values_after": {...},
    "high_risk_events": ...,
    "medium_risk_events": ...,
    "low_risk_events": ...,
    "max_magnitude": ...,
    "deepest_earthquake_km": ...,
    "top_5_regions_by_event_count": {...}
}
```

Analysis Questions

Answer the following:

1. Which region had the most earthquakes?
2. What was the strongest earthquake in the dataset?
3. How many high-risk events were detected?
4. How many earthquakes were shallow, intermediate, and deep?
5. Which day had the most earthquake activity?

Submission Requirements

Submit:

1. Python notebook/script
2. `raw_earthquakes.json`
3. `earthquakes_clean.csv`
4. `earthquakes_clean.parquet`
5. Final data quality report
6. Screenshots or outputs of analysis questions

Validation Requirement

Generate an MD5 hash of the raw API response:

```
import hashlib
import json

raw_data_hash = hashlib.md5(
    json.dumps(raw_earthquake_data, sort_keys=True).encode()
).hexdigest()

print(raw_data_hash)
```

Include this hash in your submission.

Concepts Practiced

- API ingestion
- nested JSON flattening
- Pandas DataFrames
- missing value handling
- duplicate removal
- type conversion
- string cleaning
- datetime conversion
- feature engineering
- Boolean filtering
- Parquet output
- data quality reporting

Goal

Build a production-style data cleaning pipeline that converts messy live API data into a reliable analytics-ready dataset.